Code Generation

Code generation can be considered as the final phase of compilation. Through post code generation, optimization process can be applied on the code, but that can be seen as a part of code generation phase itself. The code generated by the compiler is an object code of some lower-level programming language, for example, assembly language. We have seen that the source code written in a higher-level language is transformed into a lower-level language that results in a lower-level object code, which should have the following minimum properties:

- It should carry the exact meaning of the source code.
- It should be efficient in terms of CPU usage and memory management.

We will now see how the intermediate code is transformed into target object code (assembly code, in this case).

Peephole Optimization

This optimization technique works locally on the source code to transform it into an optimized code. By locally, we mean a small portion of the code block at hand. These methods can be applied on intermediate codes as well as on target codes. A bunch of statements is analyzed and are checked for the following possible optimization

Peephole Optimization for Code Optimization:

1. Redundant instruction elimination
2. Unreachable code
3. Flow of control optimization
4. Algebraic expression simplification
5. Strength reduction
6. Constant Folding
7. Accessing machine instructions (Machine idioms)

1. Redundant instruction elimination

At compilation level, the compiler searches for instructions redundant in nature. Multiple loading and storing of instructions may carry the same meaning even if some of them are removed. For example:

Example 1:

…..

…..

LOAD R0, a

STORE a, R0

…..

…..

Example 2:

```
Initial code:
y = x + 5;
i = y;
z = i;
w = z * 3;

Optimized code:
y = x + 5;
i = y;
w = y * 3;
```

2. Unreachable code

Unreachable code is a part of the program code that is never accessed because of programming constructs. Programmers may have accidently written a piece of code that can never be reached.

Example:

….

….

LOAD R0, a

LOAD R1, b

JMP L1

ADD R0, R1

L1:

SUB R0, R1

….

….

In the above code, “ADD R0, R1” is unreachable code and has to be deleted.

3. Flow of control optimization

There are instances in a code where the program control jumps back and forth without performing any significant task. These jumps can be removed. Consider the following chunk of code:

Example 1:

```
...
MOV R1, R2
GOTO L1
...
L1 :   GOTO L2
L2 :   INC R1
```

In this code, label L1 can be removed as it passes the control to L2. So instead of jumping to L1 and then to L2, the control can directly reach L2, as shown below:

```
...
MOV R1, R2
GOTO L2
...
L2 :   INC R1
```

4. Algebraic expression simplification

There are occasions where algebraic expressions can be made simple. For example, the expression **a = a + 0** can be replaced by **a** itself and the expression a = a + 1 can simply be replaced by INC a.

5. Strength reduction

The operators that consume higher execution time are replaced by the operators consuming less execution time. There are operations that consume more time and space. Their 'strength' can be reduced by replacing them with other operations that consume less time and space, but produce the same result.

```
Initial code:
y = x * 2;

Optimized code:
y = x + x;     or     y = x << 1;

Initial code:
y = x / 2;

Optimized code:
y = x >> 1;
```

6. Constant folding:

The code that can be simplified by user itself, is simplified.

```
Initial code:
x = 2 * 3;

Optimized code:
x = 6;
```

7. Accessing machine instructions

The target machine can deploy more sophisticated instructions, which can have the capability to perform specific operations much efficiently. If the target code can accommodate those instructions directly, that will not only improve the quality of code, but also yield more efficient results.

Example:

In the place of increment by one i.e. “ADD R0, 1”, “INC R0” can be used (if these type of automatic increment machine instruction or any sophisticated instruction is available).

Code Generator

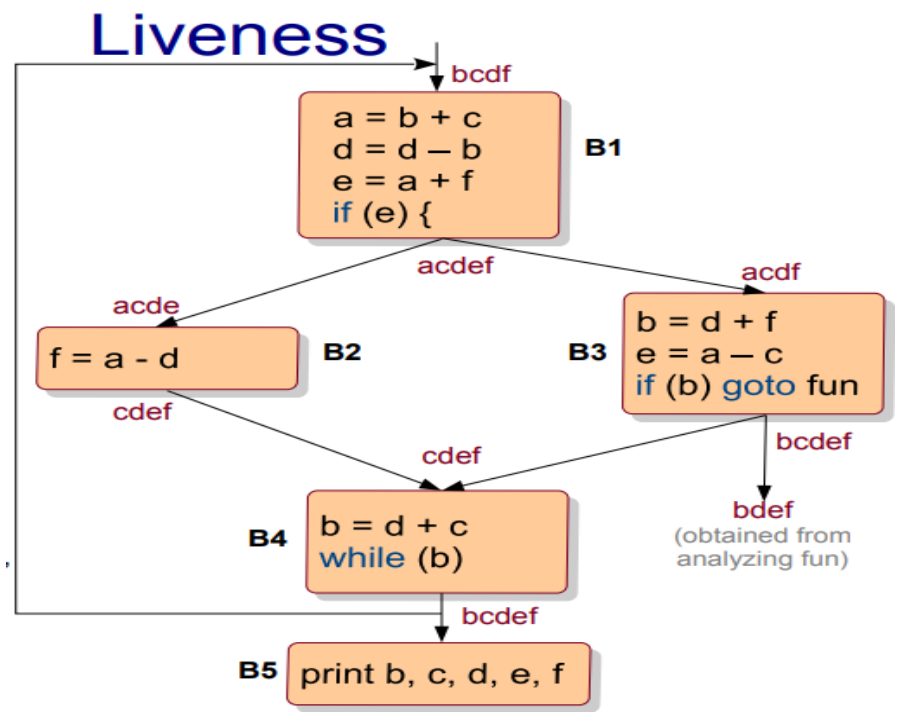
A code generator is expected to have an understanding of the target machine’s runtime environment and its instruction set. The code generator should take the following things into consideration to generate the code:

- **Target language** : The code generator has to be aware of the nature of the target language for which the code is to be transformed. That language may facilitate some machine-specific instructions to help the compiler generate the code in a more convenient way. The target machine can have either CISC or RISC processor architecture.
- **Intermediate Representation Type**: Intermediate representation has various forms. It can be in or 3-address code, syntax tree or Polish Notation.
- **Selection of instruction**: The code generator takes Intermediate Representation as input and converts (maps) it into target machine’s instruction set. One representation can have many ways (instructions) to convert it, so it becomes the responsibility of the code generator to choose the appropriate instructions wisely.
- **Register allocation**: A program has a number of values to be maintained during the execution. The target machine’s architecture may not allow all of the values to be kept in the CPU memory or registers. Code generator decides what values to keep in the registers. Also, it decides the registers to be used to keep these values.

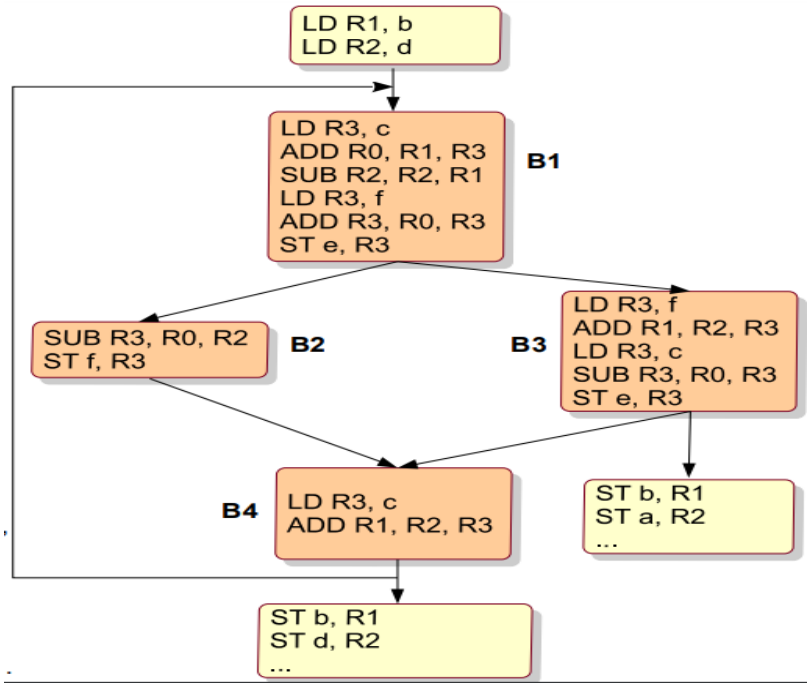
  Example :

```
while (b) {
  a = b + c
  d = d – b
  e = a + f
  if (e) {
     b = d + f
     e = a – c
     if (b) goto fun
  } else {
     f = a - d
  }
  b = d + c
}
print b, c, d, e, f
```

First-Come-First-Served way is often not the best policy for register allocation. ● We need to perform some analysis to find out the benefit of allocating registers to variables. ● We may have to assign cost / benefit to various operations within a loop. ● What if we say that K registers would be allocated to the top K variables that have the maximum number of uses?



● R1 and R2 remain assigned to b and d throughout. ● R3 is loaded repeatedly inside the loop as an auxiliary register. ● a is not live at the start, hence it is not loaded initially. ● At the end of the loop, the register values are stored back.



- **Ordering of instructions**: At last, the code generator decides the order in which the instruction will be executed. It creates schedules for instructions to execute them. Instruction order affects execution efficiency.

  Example :

  Original ordering:

```
1: R0 = a
2: R1 = b
3: R2 = c
4: R3 = R0 + R1
5: R4 = R2 + R3
6: d = R4
```

  After Re-ordering:

```
1: R0 = a
2: R1 = b
4: R2 = R0 + R1
3: R0 = c
5: R3 = R2 + R0
6: d = R3
```